

5. Instrumentation

└ 5.2 Alerting and metrics

└ 5.3.3 Distributed tracing with App Insights

-
1. What is distributed tracing and why is it critical in microservices or cloud-native architectures?
 2. How does Application Insights implement distributed tracing?
 3. What are key telemetry types collected by App Insights for distributed tracing?
 4. How do you instrument .NET/Java/Node.js applications for distributed tracing with App Insights?
 5. What is an operation ID and how does it correlate telemetry across services?
 6. How are dependencies and requests visualized in the App Insights Application Map?
 7. How can you query distributed traces in Application Insights using Kusto Query Language (KQL)?
 8. How do you set up automatic vs. manual correlation for custom telemetry in distributed tracing?
 9. What are best practices to minimize trace sampling impact and maximize troubleshooting value?
 10. How do you integrate distributed tracing from App Insights with Azure Monitor or third-party observability tools?
-

1. What is distributed tracing and why is it critical in microservices or cloud-native architectures?

Distributed tracing is tracking a request as it flows across multiple services or components. It is critical in microservices/cloud-native environments to identify bottlenecks, diagnose failures, and understand dependencies across distributed systems.

2. How does Application Insights implement distributed tracing?

Application Insights uses correlation IDs injected into telemetry (requests, dependencies, exceptions) to automatically track and link operations across components and services.

3. What are key telemetry types collected by App Insights for distributed tracing?

Key telemetry types:

- Request telemetry (incoming HTTP requests),
 - Dependency telemetry (calls to databases, HTTP, services),
 - Exception telemetry,
 - Trace telemetry (logs),
 - and Custom telemetry (events, metrics).
-

4. How do you instrument .NET/Java/Node.js applications for distributed tracing with App Insights?

Use *Application Insights* SDK or agents.

- .NET: Install the *NuGet* package or enable with *Application Insights* Agent.
 - Java: Use *Application Insights* Java agent.
 - Node.js: Use the npm package. Configuration auto-instruments most frameworks.
-

5. What is an operation ID and how does it correlate telemetry across services?

An operation ID is a unique identifier assigned to each end-to-end request. It propagates across services via headers, allowing *App Insights* to correlate telemetry events for a single operation.

6. How are dependencies and requests visualized in the App Insights Application Map?

Application Map displays detected components and dependencies as a graph, showing request flow, health, response times, and failure hotspots, enabling quick drill-down into distributed traces.

7. How can you query distributed traces in Application Insights using Kusto Query Language (KQL)?

Use the *traces*, *requests*, and *dependencies* tables in *Log Analytics*. Query by *operation_Id* to retrieve and analyze related events across services for a given distributed trace.

8. How do you set up automatic vs. manual correlation for custom telemetry in distributed tracing?

Automatic correlation is handled by the SDK/agent for supported frameworks. For custom telemetry, set the *operation_Id* and *parent_Id* explicitly when tracking events to maintain trace linkage.

9. What are best practices to minimize trace sampling impact and maximize troubleshooting value?

Use adaptive sampling to balance telemetry volume and cost. Always capture telemetry for errors/exceptions. Tune sampling rates as needed to ensure critical traces are retained for troubleshooting.

10. How do you integrate distributed tracing from App Insights with Azure Monitor or third-party observability tools?

Enable integration by configuring Diagnostic settings to export logs/metrics to *Azure Monitor* or *Log Analytics*. Use *OpenTelemetry* exporters or API integrations for third-party tools.